AI in images and video processing

Thomas de Jaeger

thomas.de-jaeger@epita.fr

Spring 2025

Intro to PyTorch + CNNs for Classification

Session 3

Intro to PyTorch + CNNs for Classification

Session 3

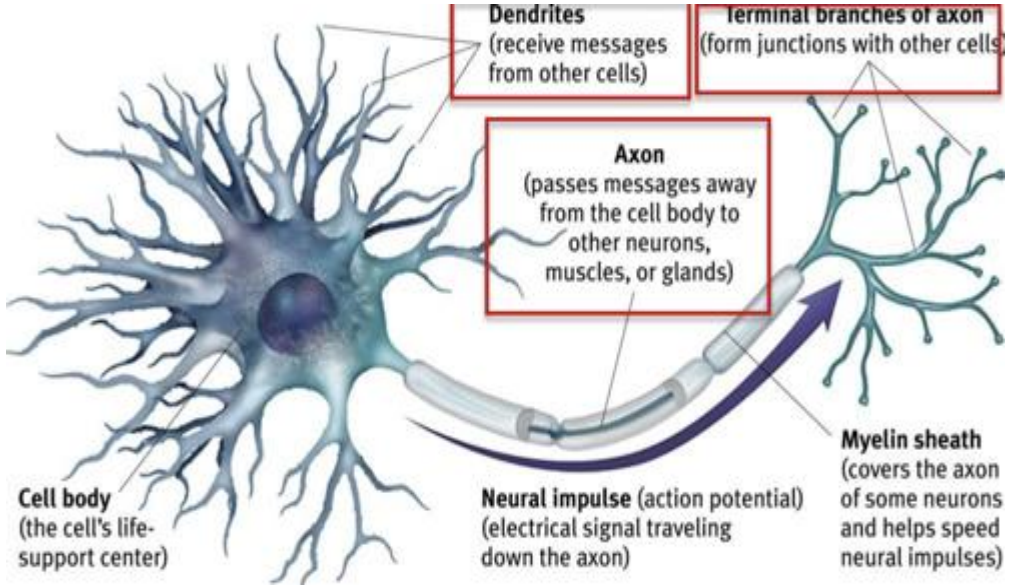
Objectives:

- Understand how artificial neural networks are inspired by the brain
- Learn how a vanilla neural network works through a complete mathematical example
- Break down the components of a CNN
- Learn to build and train CNNs using PyTorch

From Biological to Artificial Neurons

A biological neuron:

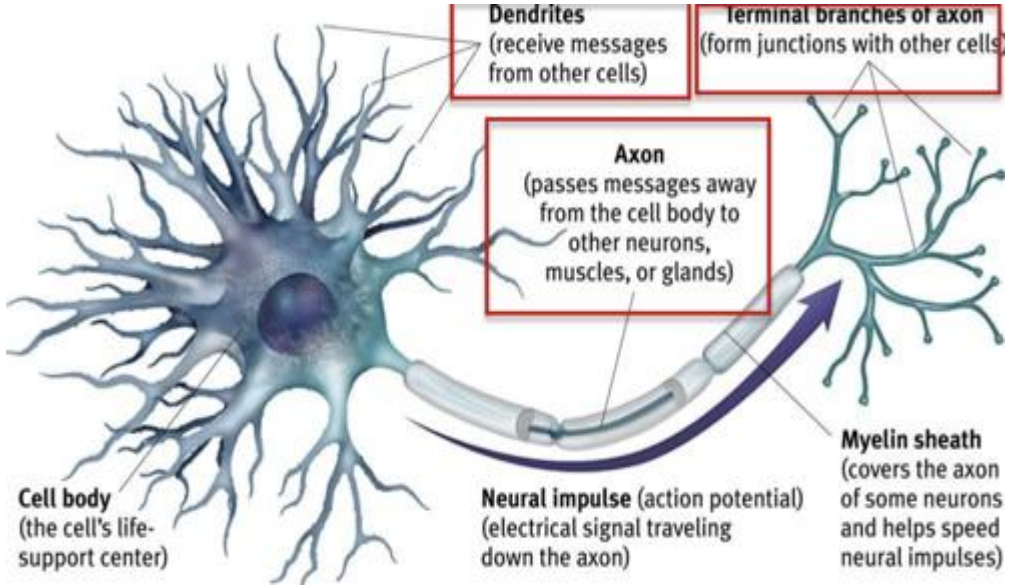
- receives signals via dendrites where each signal has a different synaptic strength (representing its importance)
- These signals are processed in the cell body, and if the **total input exceeds a certain threshold**,
- the neuron sends out an electrical signal via the axon.



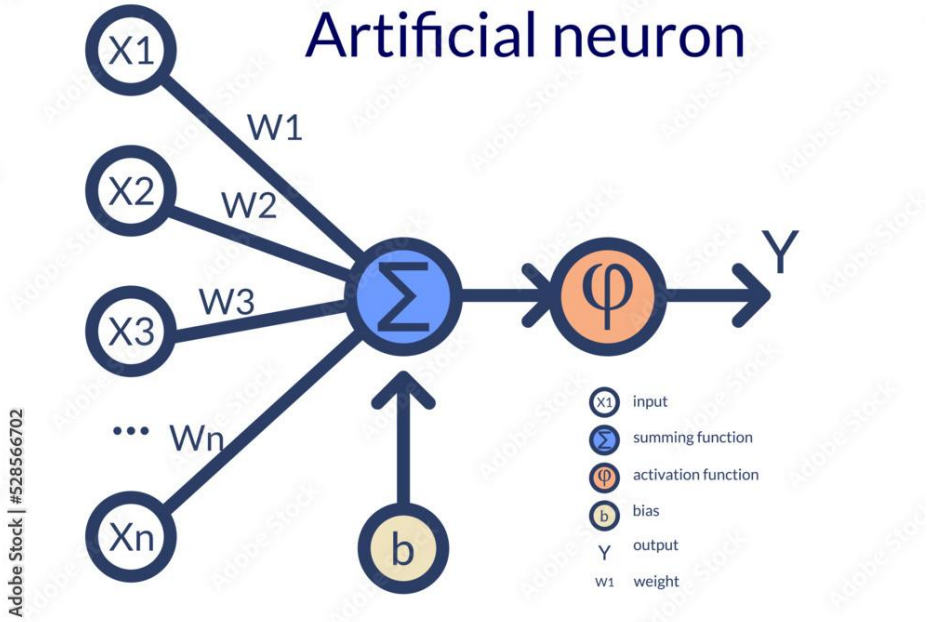
From Biological to Artificial Neurons

A biological neuron:

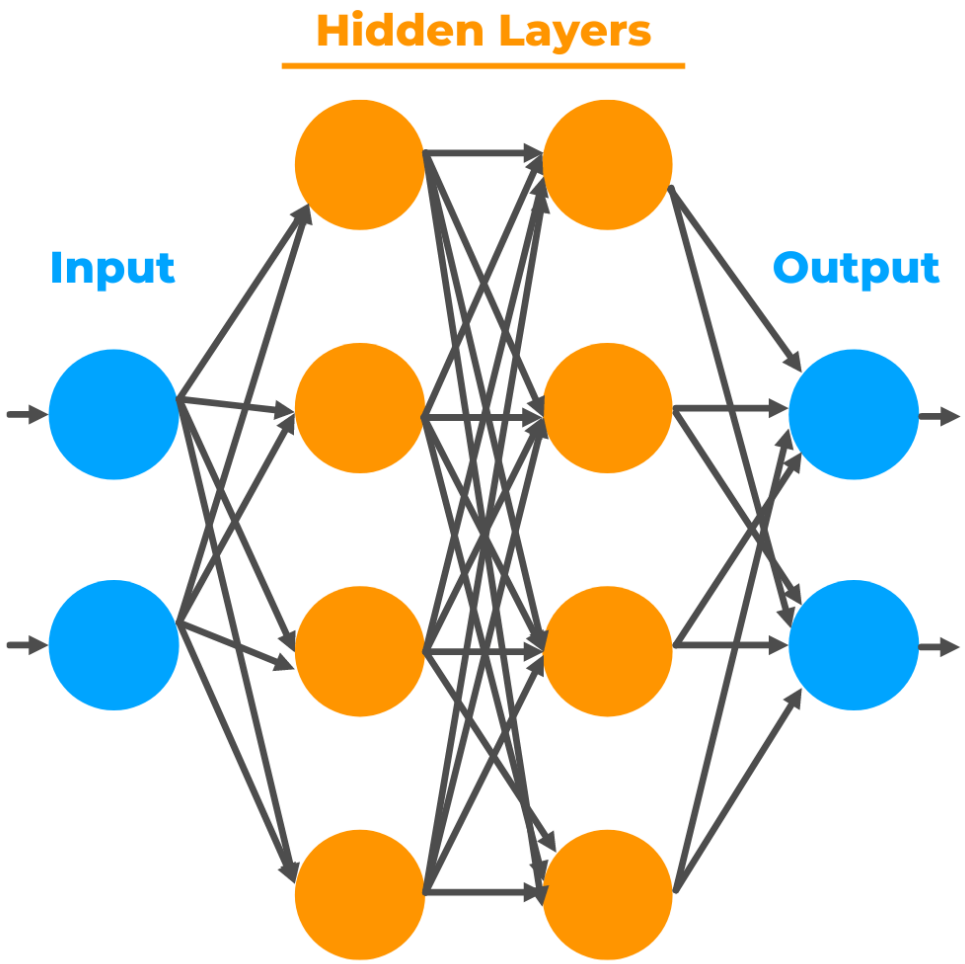
- **receives signals via dendrites** where each signal has a different **synaptic strength** (representing its importance)
- These signals are processed in the cell body, and if the **total input exceeds a certain threshold**,
- the neuron **sends out an electrical signal via the axon.**



An artificial neuron: mimic this behavior using a mathematical formula: **a weighted** sum **of inputs**, plus a bias, passed **through an activation function,** and **give an output**

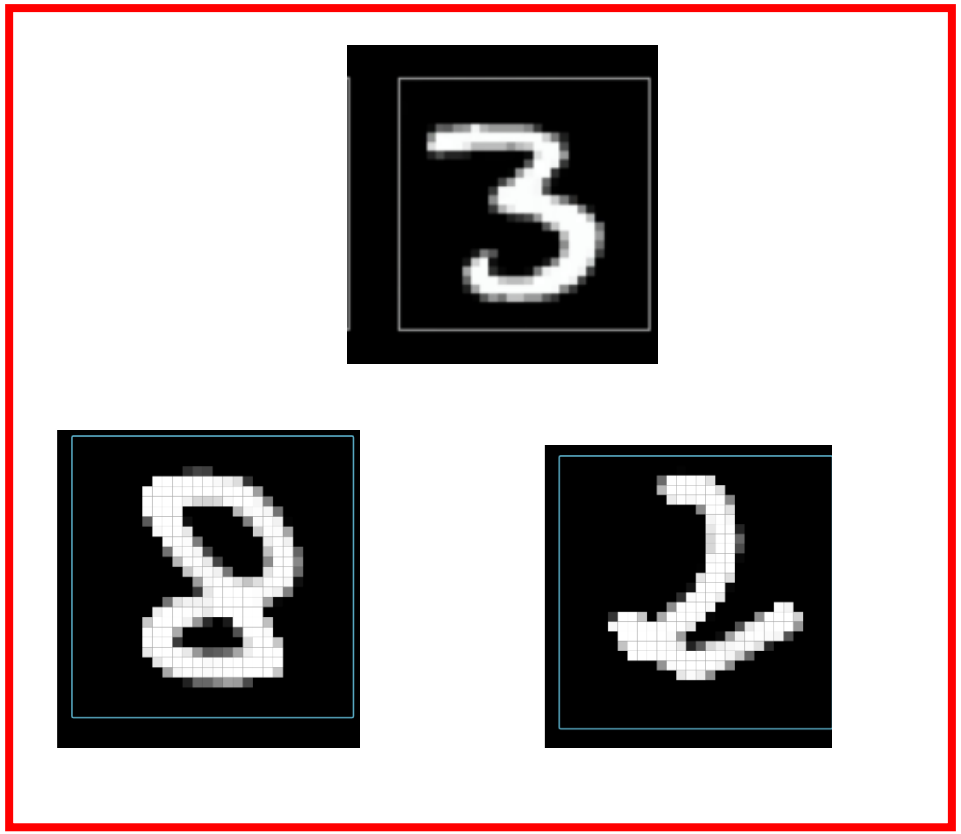


From Biological to Artificial Neurons

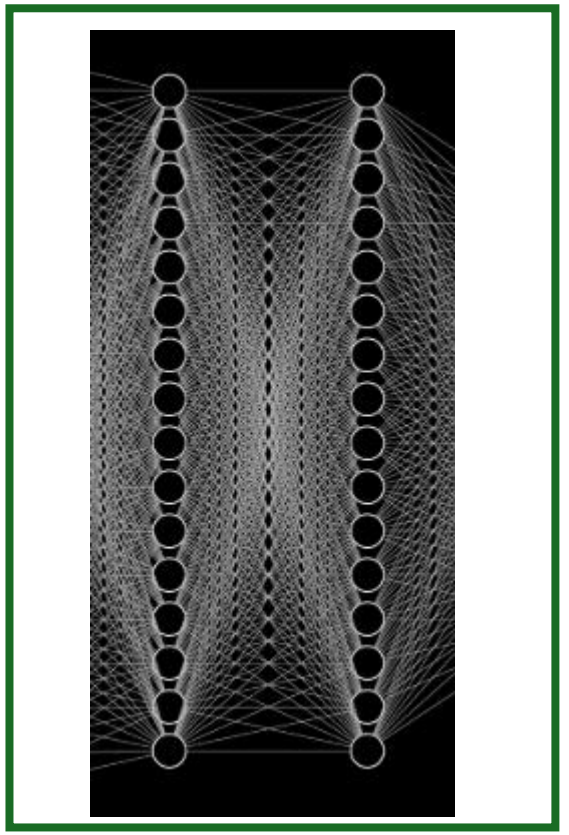


How Artificial Neural Networks work?

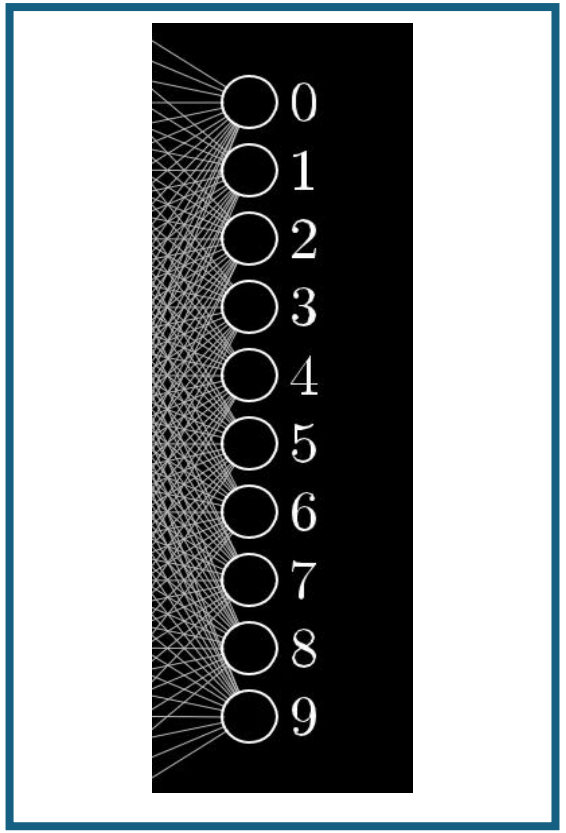
Create a NN to classify image?



INPUTS



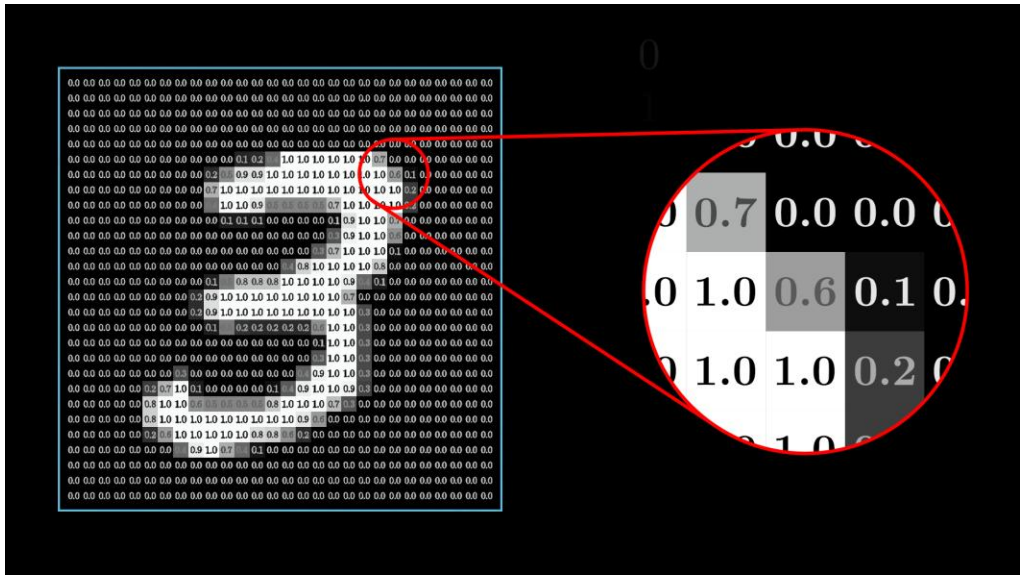
Hidden layers



OUTPUTS

How Artificial Neural Networks work?

Create a NN to classify image?

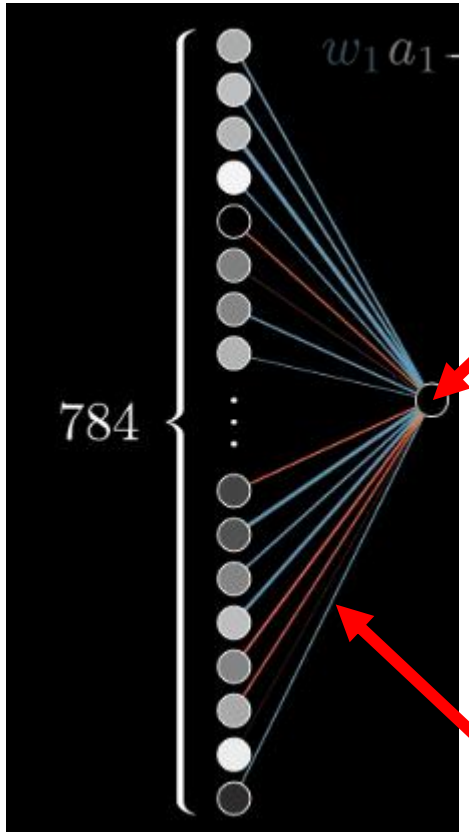


One image is:
28x28pixels= 784 pixels

784 neurons of inputs

How Does a Neuron Compute?

Inside a Single Neuron



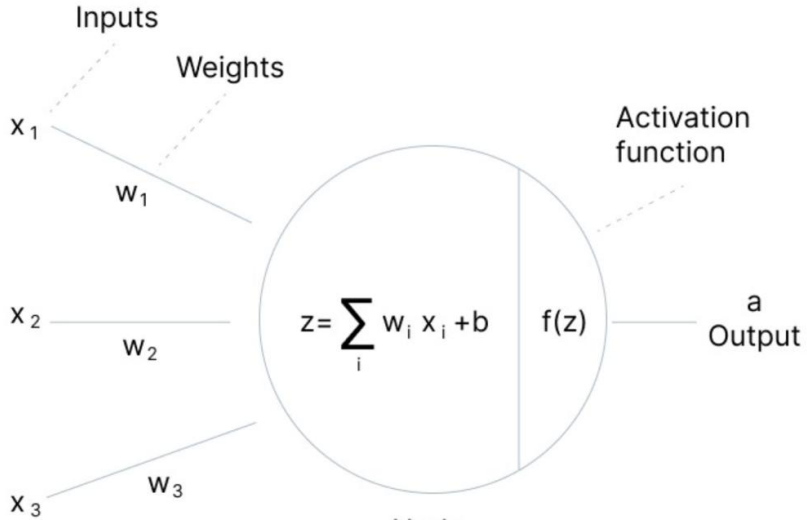
Single Neuron In the hidden layer

Weight

Inputs

Each input has a different weight (strength/importance):
w1,.....wn

The output of each neuron is calculated by applying a weighted sum of its inputs (from the previous layer), adding a bias, and then passing this sum through an activation function (f).



Weight

What Are Weights in a Neural Network?

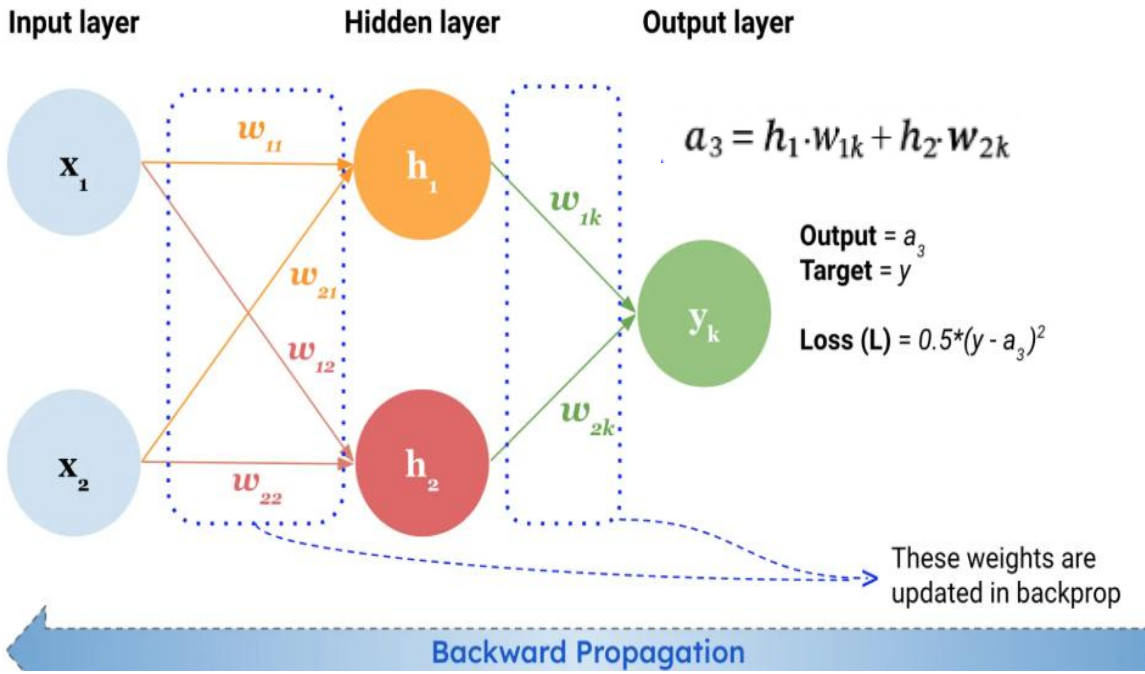
- Weights are the learnable parameters of a neural network.
- They determine how much influence each input feature has on the neuron's output.

During training, the network adjusts weights to minimize prediction error.

- A high weight = strong influence
- A zero weight = no effect

Weight pruning: is a technique used to reduce the complexity of an NN by removing weights that have little to no influence on the output, effectively setting them to zero

Importance: they directly influence the signals transmitted across the network and ultimately determine the network's output.



Bias

What is bias?

- The bias is added to the weighted sum of inputs.
- It shifts the activation function and allows neurons to activate even with zero inputs.

Why is bias important?

- Shifting the activation: Without bias, the output would always be zero when all inputs are zero, which can limit the network's ability to learn complex patterns.
- Improving flexibility: Bias allows the activation function to be shifted to better fit the data, enabling the network to model more complex relationships.

Activation function

The result of the weighted sum like this can be any number, but for this network we want the activations to be values between 0 and 1.

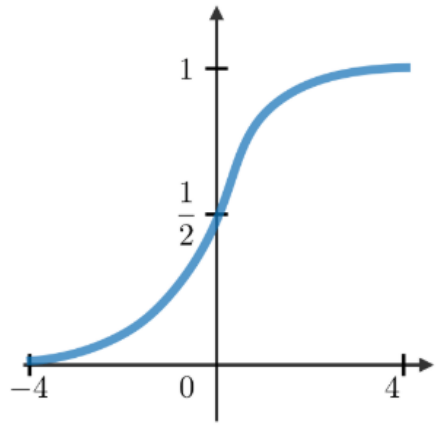
- So it's common to pump this weighted sum into some function that squishes the real number line into the range between 0 and 1

- Crucial as it introduces non-linearity into the model, allowing it to learn more complex patterns

activated if output > 0.5

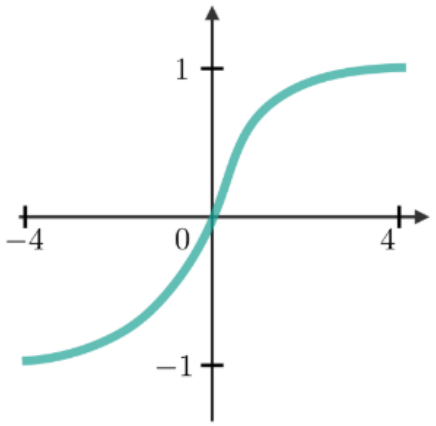
Sigmoid

$$g(z) = \frac{1}{1 + e^{-z}}$$



Tanh

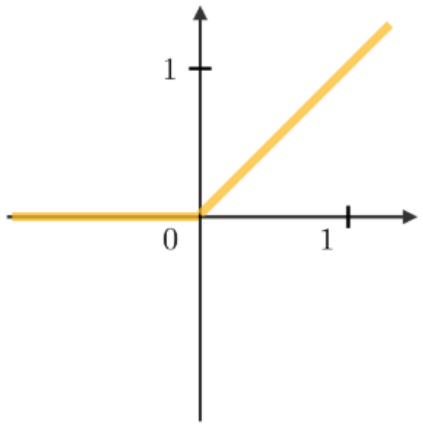
$$g(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$



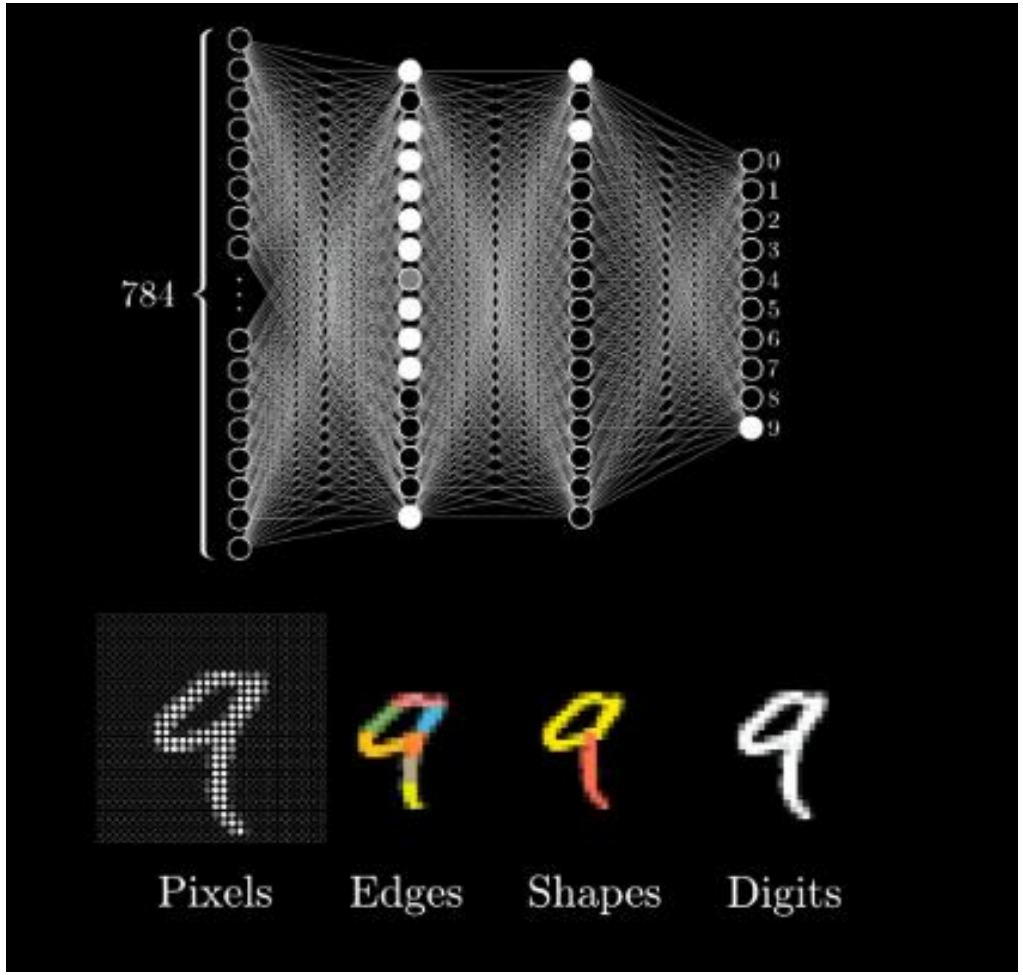
activated if output > 0

RELU

$$g(z) = \max(0, z)$$



All together



Limitation of the ANN

- Fully connected (vanilla) networks flatten the image and **ignore spatial structure**.
    - This leads to very large parameter spaces and poor generalization to spatial patterns.
- With ANN, image classification problems become difficult because 2-dimensional images need to be converted to 1-dimensional vectors.
- **Convolutional Neural Networks (CNNs) are designed for spatially structured data like images.**
- CNNs preserve spatial relationships by using small filters (kernels) that scan over the image.
- **Weight sharing and local** connectivity drastically **reduce parameters and improve generalization.**

CNNs for Classification

Convolutional Neural Networks (CNNs) are powerful architectures designed specifically for **image and spatial data**. Unlike standard neural networks that treat all inputs independently, CNNs process images by scanning small regions using **convolutional filters**. This allows them to **capture local patterns** like edges, shapes, and textures effectively.
A typical CNN is composed of the following layers:

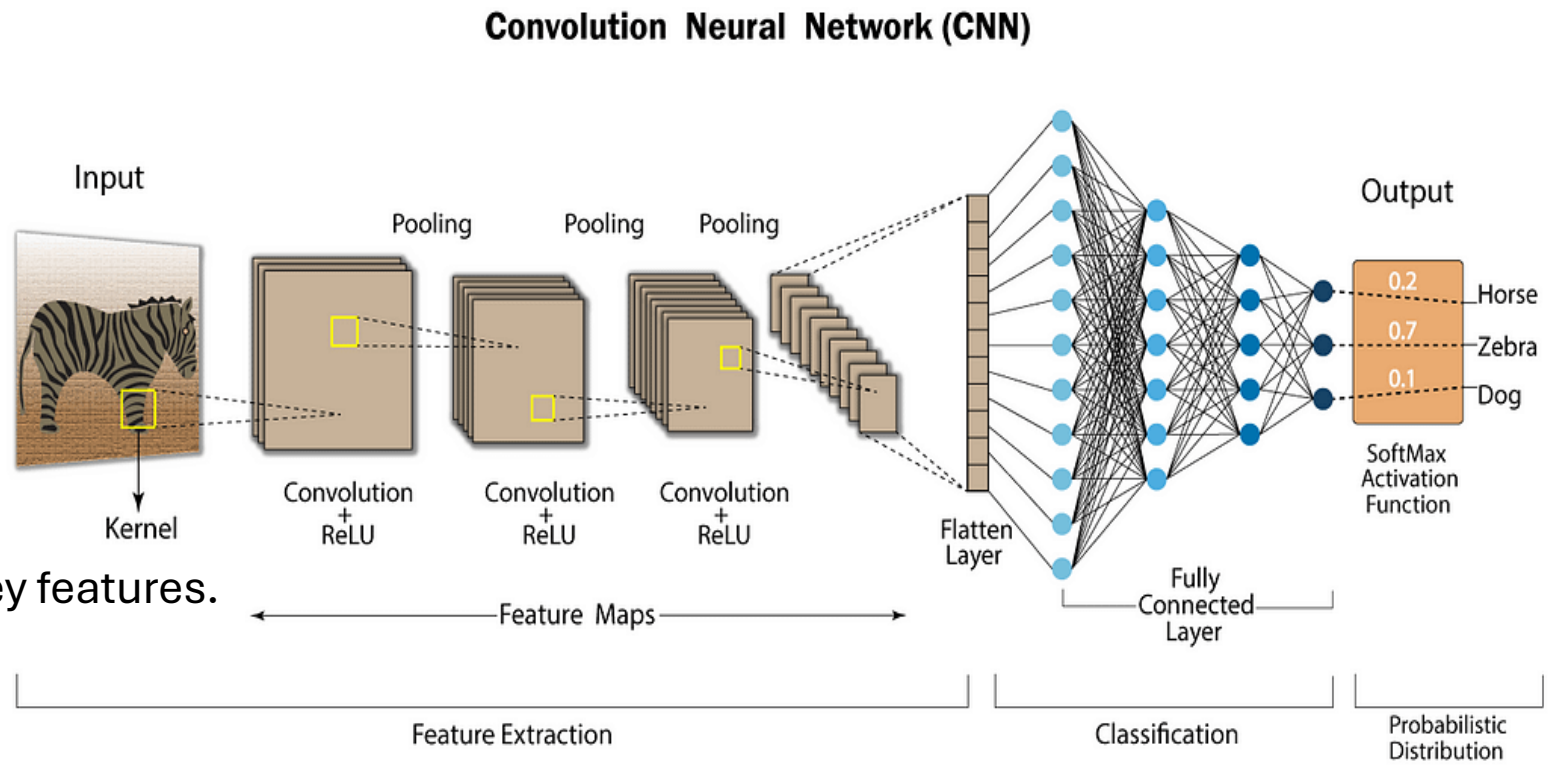
- **Input Layer:**
Receives the image.

- **Convolutional Layer**:
Applies filters to extract features.

- **Activation Function:**
Introduces non-linearity (commonly ReLU).

- **Pooling Layer:**
Reduces dimensionality and emphasizes key features.

- **Fully Connected Layer:**
Performs the final classification.



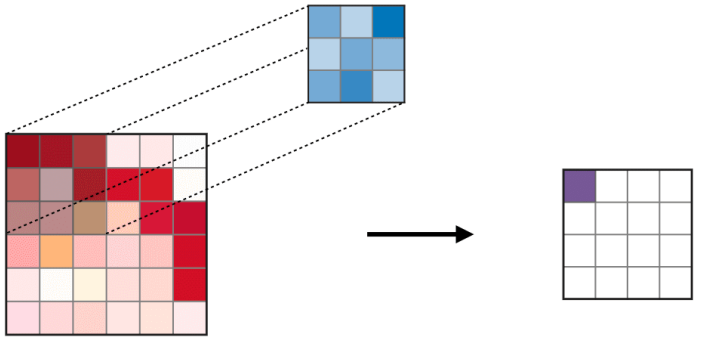
CNN: Convolutional Layer

This layer is responsible for extracting important features from the input image using a set of learnable filters/kernels (kernel are learned parameters!!)

•Applies a small learnable filter across the input image.

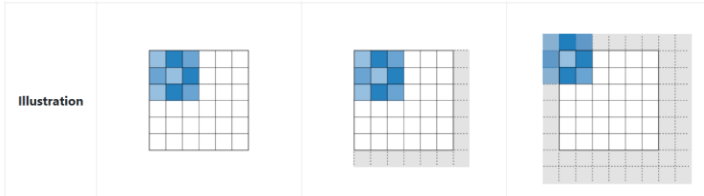
•Detects features like edges, corners, and textures.

•Uses the same weights at each location (weight sharing).



Its hyperparameters include:

- Padding (P): adds a border of zeros around the input image. This helps control the spatial size of the output and preserves edge information when applying filters.

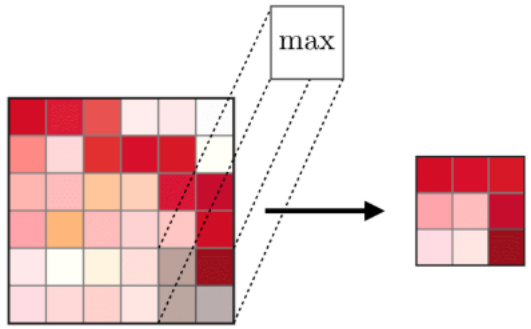
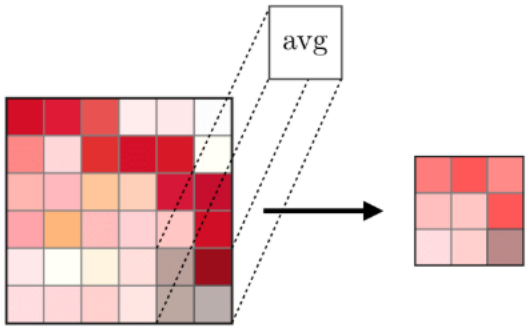


- Filter size (F): determines the spatial area each filter covers (e.g., 3×3)
- Stride (S): controls how far the filter moves across the image (e.g. 1pixel) — larger strides reduce output size

Adding an activation function to the activation layers add nonlinearity to the network

CNN: Pooling

Pooling layers follow convolution layers and are used to reduce the spatial dimensions of feature maps while retaining important information.

Type	Max pooling	Average pooling
Purpose	Each pooling operation selects the maximum value of the current view	Each pooling operation averages the values of the current view
Illustration		
Comments	• Preserves detected features • Most commonly used	• Downsamples feature map • Used in LeNet

CNN

Benefits:

- Feature extraction: Automatically learns features from raw image data, reducing manual preprocessing.
- Spatial invariance: Can recognize patterns regardless of their location, orientation, or size.
- Robust to noise: Performs well even when input images are cluttered or noisy.
- Transfer learning: Pre-trained CNNs can be adapted to new tasks with less data and compute.
- High performance: Achieves state-of-the-art results in tasks like classification, detection, and segmentation.

Limitations:

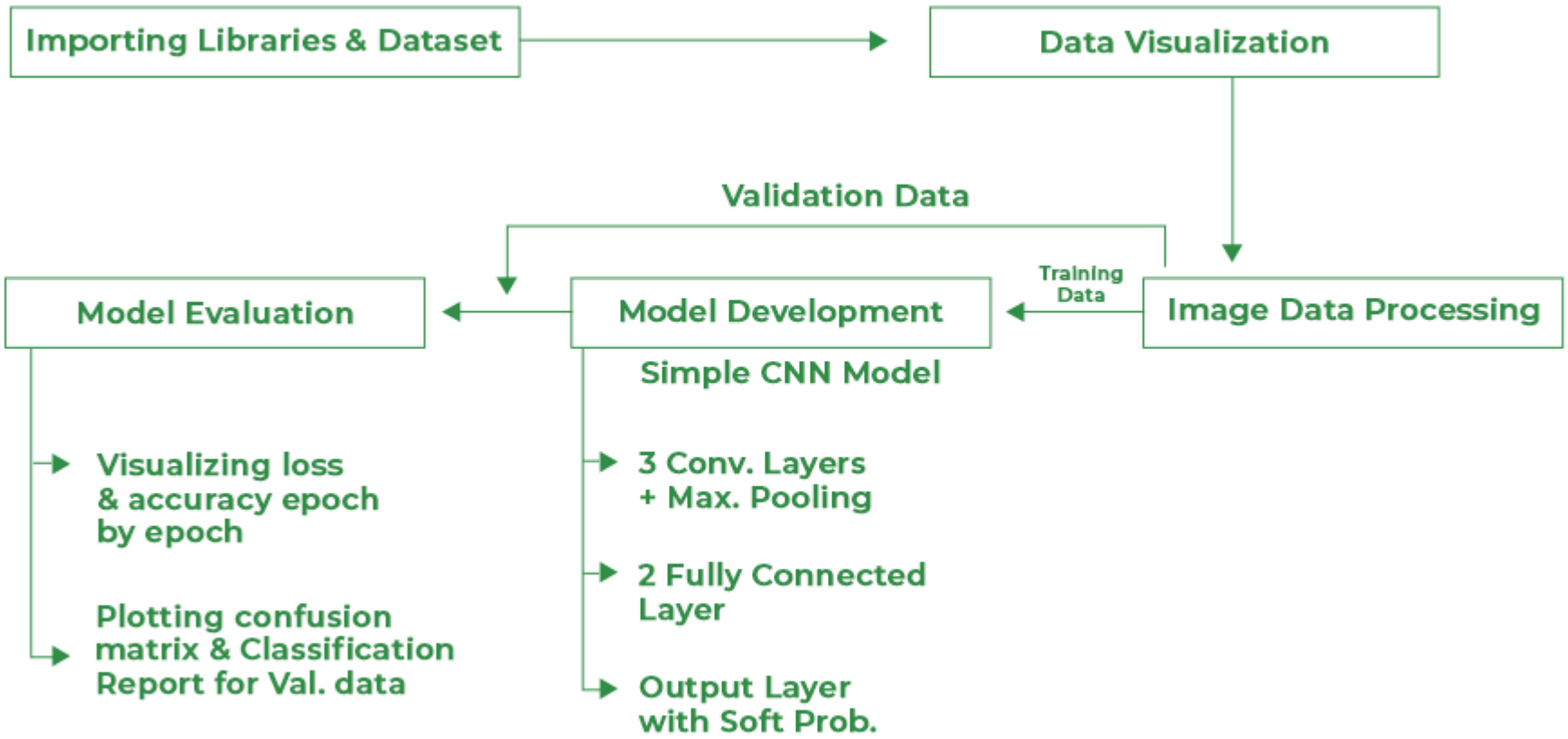
- Computational cost: Training deep CNNs can require a lot of resources and time.
- Overfitting: Prone to memorizing training data when datasets are small.
- Lack of interpretability: It's difficult to explain how or why a CNN made a specific decision.
- Structured input required: Works only with grid-like data (e.g., images), not irregular formats

Training a CNN

We will do the following steps in order:

- Load data (for pre defined dataset use torchvision
- Clean your data: normalization, augmentation, resize, outlier rejection, etc
- Define a Convolutional Neural Network (number of con2D, pooling)
- Define a loss function to compare predictions to true labels
- Train the network on the training data
  - Compute the gradients of the loss with respect to the model parameters (weights and biases) using backpropagation.
  - Adjust the weights and biases using an optimizer (e.g., SGD, Adam) to minimize the loss.
  - Loop over all batches for multiple epochs until the model performance stabilizes.
- Test the network on the test data

Training a CNN



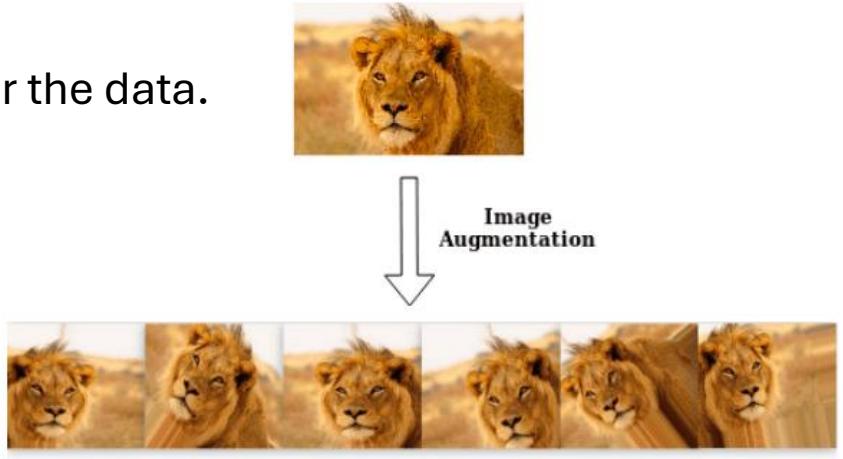
Overfitting

When a model learns the training data too well — including noise and details that don't generalize.

- Results in good training accuracy but poor performance on unseen (test) data.

How to prevent or reduce overfitting?

- More training data: Exposes the model to more variations.
- Data augmentation: Randomly change training images (rotate, flip, crop, etc.).
- Regularization: Add penalty terms to the loss (e.g., L2 weight decay).
- Dropout: Randomly disable some neurons during training.
- Early stopping: Stop training when validation loss starts to rise.
- Batch normalization: Helps by regularizing and stabilizing training.
- Simpler model: Use fewer layers or neurons if the model is too complex for the data.



Batch normalization

Applied after convolutional or fully connected layers:

Normalizes intermediate activations to zero mean and unit variance

Why?

- **Stabilizes training**: reduces fluctuations in layer outputs, helping the model learn more consistently.
- **Reduces internal covariate shift**: Keeps the input distribution of each layer more stable during training.
- **Allows higher learning rates**: Makes training more robust to large updates, speeding up convergence.
- **Acts as a form of regularization**: Adds slight noise from mini-batch stats, which helps prevent overfitting.

Recommended in most CNNs for audio/image tasks

Introduction to Pytorch

PyTorch is a powerful open-source deep learning framework developed by Meta AI. It is widely used in research and industry for building and training neural networks.

Why?

- Dynamic computation graphs: More intuitive and flexible than static graphs (more intuitive debugging)
- Pythonic and easy to debug: Works seamlessly with NumPy and native Python tools
- Extensive GPU support: Easily switch between CPU and GPU with .to(device)
- Strong community and ecosystem: Includes torchvision, torchaudio, and PyTorch Lightning
- Widely adopted: Used in academia, industry, and production systems.

LAB

IA_images_Lab3_questions (lab/lecture Pytorch)